Short-answer questions

1. Why is `BCEWithLogitsLoss` usually preferred over applying `sigmoid` yourself and then using BCE?
2. Why should masked-out tokens get a very **negative** attention score before softmax?
3. What is wrong with taking softmax across `dim=0` here?
4. Why does forgetting `optimizer.zero_grad()` matter?
5. How would you identify potentially noisy labels after training?
6. Besides accuracy, what metric would you report if classes were meaningfully imbalanced?

Solution

Main bugs to fix

1. **Normalization leakage / padding issue**
   The buggy code normalizes using the whole dataset, including validation data and padded positions. Better: compute stats from **train only** and only over **valid tokens**.
2. **Wrong mask direction**
   Padding tokens should receive a very negative value like `-1e9`, not `+1e9`, so softmax assigns them near-zero weight.
3. **Wrong softmax dimension**
   Attention should normalize across tokens within each example: `dim=1`, not across the batch.
4. **Double sigmoid**
   `BCEWithLogitsLoss` expects raw logits. Do **not** apply `sigmoid` in `forward`.
5. **Missing gradient reset**
   Need `optimizer.zero_grad()` each step or gradients accumulate across minibatches.
6. **Validation in train mode**
   Use `model.eval()` for validation. With dropout enabled, leaving train mode makes evaluation noisy and wrong.
7. **Broken accuracy threshold**
   Threshold should be `0.5` on probabilities, not `0.0`.